

Agenda

- Background
- Tasks assigned
- Generating SBOMs & Results
 1. **Trivy**
 2. **Anchore sbom action**
 3. **Cdxgen**
- Future directions

Literature Review

-The Impact of SBOM Generators on Vulnerability Assessment in Python: A Comparison and a Novel Approach:

- The paper sought two RQs:
 - (i) To what extent does the SBOM generation process affect vulnerability detection in Python projects
 - (ii) How can the SBOM generation approach be improved for better security assessment accuracy?
- PIP-sbom: **64%** improvement in precision and a **59%** increase in recall

Trivy SBOM Testing

- Using aquasecurity/trivy-ci-test
- Found Total 48 vulnerabilities: UNKNOWN: 0, LOW: 1, MEDIUM: 26, HIGH: 16, CRITICAL: 5

Trivy Results

Total: 5 (CRITICAL: 5)

Library	Vulnerability Title	Severity	Status	Installed Version	Fixed Version	
django	CVE-2019-19844 email address allows account takeover	CRITICAL	fixed	2.0.9	1.11.27, 2.2.9, 3.0.1	Django: crafted https://avd.aquasec.com/nvd/cve-2019-19844
	CVE-2020-7471 al SQL injection via StringAgg(delimiter)				1.11.28, 2.2.10, 3.0.3	django: potenti https://avd.aquasec.com/nvd/cve-2020-7471
pyyaml	CVE-2019-20477 execution through python/object/apply FullLoader			5.1	5.2	PyYAML: command constructor in https://avd.aquasec.com/nvd/cve-2019-20477
	CVE-2020-14343				5.4	PyYAML: incompl

Anchore SBOM Action

- Employed anchore sbom generation on workspace of <https://github.com/abdullahsohail/abdullahsohail.github.io>
- Tool used was syft-1.16.0

```
{
  "spdxVersion": "SPDX-2.3",
  "dataLicense": "CC0-1.0",
  "SPDXID": "SPDXRef-DOCUMENT",
  "name": ".",
  "documentNamespace": "https://anchore.com/syft/dir/9907fa3a-45e0-4029-8954-7bbd7ea2e7e2",
  "creationInfo": {
    "licenseListVersion": "3.25",
    "creators": [
      "Organization: Anchore, Inc",
      "Tool: syft-1.16.0"
    ],
    "created": "2024-11-08T01:38:45Z"
  }
},
```

Anchore SBOM Cont'd

```
"packages": [  
  {  
    "name": "actions/checkout",  
    "SPDXID": "SPDXRef-Package-github-action-actions-checkout-529af9067153091f",  
    "versionInfo": "v4",  
    "supplier": "NOASSERTION",  
    "downloadLocation": "NOASSERTION",  
    "filesAnalyzed": false,  
    "sourceInfo": "acquired package info from GitHub Actions workflow file or composite action file: /.github/workflows/sbom-g",  
    "licenseConcluded": "NOASSERTION",  
    "licenseDeclared": "NOASSERTION",  
    "copyrightText": "NOASSERTION",  
    "externalRefs": [  
      {  
        "referenceCategory": "SECURITY",  
        "referenceType": "cpe23Type",  
        "referenceLocator": "cpe:2.3:a:actions\\/\\/checkout:actions\\/\\/checkout:v4:*:*:*:*:*:*"   
      },  
      {  
        "referenceCategory": "PACKAGE-MANAGER",  
        "referenceType": "purl",  
        "referenceLocator": "pkg:github/actions/checkout@v4"   
      }  
    ]  
  },  
]
```

Cdxgen sbom generation

```
{
  "bomFormat": "CycloneDX",
  "specVersion": "1.6",
  "serialNumber": "urn:uuid:f1e1b7f2-7c44-4de2-acc8-7f27867795c7",
  "version": 1,
  "metadata": {
    "timestamp": "2024-11-08T01:01:32Z",
    "tools": {
      "components": [
        {
          "group": "@cyclonedx",
          "name": "cdxgen",
          "version": "10.11.0",
          "purl": "pkg:npm/%40cyclonedx/cdxgen@10.11.0",
          "type": "application",
          "bom-ref": "pkg:npm/@cyclonedx/cdxgen@10.11.0",
          "publisher": "OWASP Foundation",
          "authors": [
            {
              "name": "OWASP Foundation"
            }
          ]
        }
      ]
    }
  }
}
```

Future directions

- Literature Review
- Further sbom tools' testing
- LLMs employed to predict dependency accuracy issues or assist in automatically identifying dependency relationships that are prone to false positives or negatives.

Meeting 22/11/2024:

Agenda

- Background
- Tasks assigned
- Generating SBOMs & Results
- Future directions

Salus

- Securing Scanners and CVE
- Deploying a Scanner
 - Manually do it (every time the code changes)
 - Run Scanner in the CI/CD pipeline via the repo's test suite (push requests)

Issue: M scanners and N codebases → M x N code changes

Enter Salus

- Run Latest Salus Container
- **Command:** `docker run --rm -t -v "$(pwd):/home/repo" coinbase/salus`
- How Salus works:

Ruby app in container:

- Initializes with configuration (more on this later)
- Loops through each scanner
 - Ruby app? → run `bundle-audit check`
 - Rails app? → run `brakeman`
 - Node app? → run `npm audit`
- Compiles report, prints to STDOUT and HTTP post
- Exits !0 if issues are found (which could fail CI)

Sample Output

```
$ cd /path/to/repo
```

```
$ docker run --rm -t -v $(pwd):/home/repo coinbase/salus
```

```
##### Salus Scan v1.0.0 for Project #####
```

```
BundleAudit => passed
```

```
NPMAudit => passed
```

```
PatternSearch => passed
```

```
RepoNotEmpty => passed
```

```
ReportNodeModules
```

```
ReportRubyGems
```

```
overall => passed
```

Meeting: 16th January 2025

SBOM Signing – CycloneDX-CLI

Objectives and Focus

Primary Objective: Investigate the security guarantees, assumptions, and utility of SBOM signing features.

Key Goals:

- Understand signature generation, validation, and use cases.
- Simulate real-world signing scenarios, e.g., missing/incompatible signatures.
- Analyze security guarantees in dependency chains.

SBOM Tool Under Study: CycloneDX-CLI

- Representative case study: **NodeGoat Project**
- Excluded projects: OWASP Juice Shop & VulnerableApp (due to missing package-lock.json).

Key Findings and Process

- **Signing Process:**
 - **RSA Key Pair:** Private key signs SBOM, public key verifies integrity.
 - Signature embedded in `<Signature>` tag of XML file.
- **Verification Workflow:**
 - Verifies SBOM tamper-proofness using public key.
 - Compares original hash with decrypted signature hash.
- **Key Observations:**
 - Any SBOM modification (e.g., added lines) invalidates signature.
 - **Scenario Examples:**
 - **Basic Use Case:** Signed and verified SBOM with no issues.
 - **Tampered SBOM:** Detected alteration; signature verification failed. Upon reverting, verification passed.
 - **Signing with Different Private Keys:** Failure (as expected) as incorrect key used.
 - **Different Signature Format:** (to be tested)

Simulated Scenarios and Future Work

Dependency Chain Simulations:

- Created mock libraries **x**, **y**, **z**:
 - **z** → depends on **y**, **y** → depends on **x**.
 - Tested integrity verification through the chain.
- **Scenario**: Missing signatures in the chain to analyze impact.

Next Steps:

- Refine use-case scenarios to address assumptions and gaps.
- Test more complex dependency graphs and integration points.